# Crypto – TME – Notes

## FLAG 1 : Déchiffrement symétrique (AES)

### Objectif :

- Reconstruire la fonction inverse « decrypt() » de la fonction « encrypt() » donnée, puis de l'utiliser sur le ciphertext donnépour le déchiffrer avec un Password donné.

### Contexte :

- J'ai :
  - Une fonction « encrypt() » qui chiffre n'importe quel Plaintext en Ciphertext.
- Je connais :
  - Un Ciphertext en AES-128-CBC codé en base64 (A-Z, a-z, 0-9,+ /).
  - Un mot de passe : « ISECR0XX ».

### Méthode :

- A partir de la fonction « encrypt() » donnée, je dois écrire la fonction inverse « decrypt() » pour déchiffrer le Ciphertext donné avec le Password donnée.
- Lancer le code « decrypt_ciphertext_post_it.py ».

### Explications du code :

- Déchiffrer avec « aes-128-cbc », c'est :
  - Utiliser l'algorithme de base AES.
  - Avec une clé de taille 128 bits.
  - Selon le mode d'opération CBC.
- Réutiliser la même fonction de dérivation de clé « PBKDF2 », autrement la clé et l'IV reconstruits ne seront pas les bons, donc le déchiffrement échouera.

### Résultat :

- J'obtiens le Plaintext associé au Ciphertext donné.

**FLAG 2 : Chiffrement asymétrique (RSA)**

**Objectif :**

- Chiffrer un Plaintext avec une clé publique.

**Contexte :**

- J'ai :
    - Une clé publique de « pki_tutorial ».
    - Un Plaintext « I got it ! ».

**Méthode :**

- Récupérer la clé publique de « pki_tutorial ».
- Lancer le code « encrypt_plaintext_with_pk.py ».
    - Stocker le Plaintext « I got it! » dans un fichier texte.
    - Lancer la commande OpenSSL qui chiffre ce Plaintext avec la clé publique de pki_tutorial.
    - Convertir la sortie binaire en hexadécimal.
    - Stocker la sortie dans un fichier .hex.

**Explications du code :**

- Convertir la sortie binaire en hexadécimal.
- Supprimer les retours à la ligne pour obtenir une chaîne compacte.

**Résultat :**

- J'obtiens le Ciphertext associé au Plaintext « I got it! » donné, chiffré avec la clé publique de pki_tutorial, puis converti en hexadécimal pour pouvoir l'utiliser plus facilement.

**FLAG 3 : Créer une paire de clé publique/privée RSA**

**Objectif :**

- Générer ma propre paire de clé RSA publique/privée.

**Méthode :**

- Lancer le code « generate_pk_sk_rsa.py » :
    - Exécuter une commande OpenSSL qui génère une clé privée RSA de 2048 bits.
    - Stocker cette clé privée dans ./Keys/PrivateKeys/my_sk.pem.
    - Exécuter une seconde commande OpenSSL qui lit cette clé privée et en extrait la clé publique associée.
    - Stocker cette clé publique dans ./Keys/PublicKeys/my_pk.pem.
    - Vérifier que les deux fichiers ont bien été générés.
- Upload la clé publique obtenue sur ma carte étudiante.

**Explications du code :**

- L'option -algorithm RSA précise que l'algorithme utilisé est RSA.
- L'option -pkeyopt rsa_keygen_bits:2048 impose une taille de clé de 2048 bits.
- L'option -pubout indique qu'on veut écrire uniquement la clé publique.

**Résultat :**

- J'obtiens ma propre paire de clés RSA :
    - une clé privée ./Keys/PrivateKeys/my_sk.pem
    - une clé publique ./Keys/PublicKeys/my_pk.pem

**FLAG 4 : Signature RSA**

**Objectif :**

- Signer un challenge avec ma clé privée RSA.

**Contexte :**

- J'ai :
    - Un challenge clair.

**Méthode :**

- Lancer le code « sign_challenge_sk_rsa.py » :
    - Écrire le challenge « twerp reply visor ahead churl » dans un fichier texte.
    - Exécuter une commande OpenSSL qui calcule un haché SHA-256 du challenge et le signe avec ma clé privée RSA.
    - Stocker la signature binaire dans ./Binary/sign_challenge1.bin.
    - Convertir la signature binaire en hexadécimal.
    - Supprimer les retours à la ligne pour obtenir une chaîne compacte.
    - Stocker le résultat dans ./Hexa/sign_challenge1.hex.

**Explications du code :**

- L'option -sha256 précise que la fonction de hachage utilisée est SHA-256.
- Le script enlève les sauts de ligne de la sortie hexadécimale avec replace("\n", "") pour obtenir une chaîne continue, plus facile à copier ou à soumettre.

**Résultat :**

- J'obtiens la signature du challenge « twerp reply visor ahead churl », produite avec ma clé privée RSA, puis convertie en hexadécimal pour pouvoir l'utiliser plus facilement.

**FLAG 5 : Attaque dictionnaire par force brute sur un mot de passe AES**

**Objectif :**

- Retrouver le mot de passe utilisé pour chiffrer un Ciphertext AES par attaque sur dictionnaire.

**Contexte :**

- Je connais :
  - L'identifiant d'un utilisateur.
- J'ai :
  - Un Plaintext que l'utilisateur a chiffré avec sa clé privée.
  - Le Ciphertext associé à ce Plaintext.
  - Une liste très grande contenant les mots de passe possible de cet utilisateur.

**Méthode :**

- Lancer le code « decrypt_ciphertext_brute_force.py » :
  - Définir un Ciphertext chiffré nommé CHALLENGE.
  - Définir un Plaintext cible nommé TARGET égal à « trees lever trims loxes filly ».
  - Ouvrir le fichier ./Texts/list_keys.txt contenant une liste de mots de passe possibles.
  - Parcourir ce fichier ligne par ligne pour tester chaque mot de passe.
  - Déchiffrer le Ciphertext avec chaque mot de passe à l'aide de la fonction decrypt().
  - Comparer le résultat obtenu avec le Plaintext cible.
  - Dès qu'un mot de passe produit le bon Plaintext, l'afficher, puis arrêter la recherche.

**Explications du code :**

- Le fichier list_keys.txt est lu ligne par ligne avec enumerate(f), ce qui permet aussi d'afficher une progression.
- La ligne word = line.strip() supprime les espaces et retours à la ligne autour du mot de passe testé.
- La condition if not word: continue permet d'ignorer les lignes vides.
- La condition if out is not None and out.strip() == TARGET: vérifie à la fois que le déchiffrement a réussi et que le plaintext obtenu correspond exactement à la cible.

**Résultat :**

- J'obtiens le mot de passe correct en testant les mots de passe d'une liste jusqu'à trouver celui qui permet de déchiffrer le Ciphertext en « trees lever trims loxes filly ».

**FLAG 6 : Générer une clé RSA contenant une chaîne précise dans la clé publique**

**Objectif :**

- Générer une paire de clés RSA dont la clé publique contient la chaîne « +++ATRIUM+++ ».

**Méthode :**

- Lancer le code « build_sk_pk_atrium.py » :
    - Décoder la chaîne cible +++ATRIUM+++ depuis le Base64 vers des octets binaires.
    - Ajouter l'octet « 0x01 » pour forcer un exposant RSA impair.
    - Convertir les octets obtenus en entier.
    - Ajouter un très grand nombre à cet entier pour obtenir une grande valeur exploitable comme exposant public RSA.
    - Exécuter une commande OpenSSL qui génère une clé privée RSA de 2048 bits en imposant cet exposant public.
    - Stocker cette clé privée dans ./Keys/PrivateKeys/sk_atrium.pem.
    - Exécuter une seconde commande OpenSSL qui extrait la clé publique associée.
    - Stocker cette clé publique dans ./Keys/PublicKeys/pk_atrium.pem.
    - Lire le fichier PEM de la clé publique et vérifier que la chaîne « +++ATRIUM+++ » apparaît bien dans sa partie Base64.

**Explication du code :**

- La ligne base64.b64decode(TARGET) décode cette cible en octets binaires.
- Le script ajoute ensuite b"\x01" pour garantir que l'exposant public RSA soit impair, ce qui est une propriété nécessaire pour un exposant RSA valide.
- Le script ajoute $256 * 10 ** 100$ pour "gonfler" la valeur finale et obtenir un exposant très grand.
- La commande OpenSSL genpkey utilise ensuite -pkeyopt rsa_keygen_pubexp:<valeur> pour imposer cet exposant public RSA.
- L'option -pkeyopt rsa_keygen_bits:2048 impose une taille de clé de 2048 bits.
- L'option -pkeyopt rsa_keygen_primes:2 demande une clé RSA classique construite avec deux nombres premiers.
- Après génération, la commande openssl pkey -pubout extrait la clé publique depuis la clé privée.

**Résultat :**

- J'obtiens une paire de clés RSA :
    - Une clé privée ./Keys/PrivateKeys/sk_atrium.pem
    - Une clé publique ./Keys/PublicKeys/pk_atrium.pem contenant la chaîne « +++ATRIUM+++ » dans sa partie Base64.

**FLAG 7 : Connexion CHAP & Déchiffrement AES avec mot de passe connu**

**Objectif :**

- M'authentifier avec le protocole CHAP (Challenge-Handshake Authentication Protocol).
- Déchiffrer un Ciphertext AES à l'aide d'un mot de passe connu.

**Méthode :**

- Id = AdrienPanguel
- Mdp = __CHAP__
- Lancer le code « decrypt_ciphertext_challenge_CHAP.py » :
    - Définir le Ciphertext chiffré dans une variable ciphertext.
    - Définir le mot de passe associé dans une variable password égale à Didi-03?/.
    - Appeler la fonction « decrypt() » pour lancer le déchiffrement.
    - Stocker le résultat dans la variable plaintext.
    - Afficher le plaintext déchiffré.

**Résultat :**

- J'obtiens le Plaintext associé au Ciphertext donné.

**FLAG 8 : Connexion PK & Signature RSA**

**Objectif :**

- M'authentifier avec le protocole à clef publique.
- Signer un challenge avec ma clé privée RSA.

**Méthode :**

- Id = AdrienPanguel
- Mdp = __PK__
- Lancer le code « sign_challenge_PK_rsa.py » :
    - Écrire le challenge « fader scrub chins divvy amass » dans un fichier texte.
    - Exécuter une commande OpenSSL qui calcule un haché SHA-256 du challenge et le signe avec ma clé privée RSA.
    - Stocker la signature binaire dans ./Binary/sign_challenge_PK.bin.
    - Convertir la signature binaire en hexadécimal.
    - Supprimer les retours à la ligne pour obtenir une chaîne compacte.
    - Stocker le résultat dans ./Hexa/sign_challenge_PK.hex.

**Explications du code :**

- L'option -sha256 précise que la fonction de hachage utilisée est SHA-256.
- Le script enlève les sauts de ligne de la sortie hexadécimale avec replace("\n", "") pour obtenir une chaîne continue, plus facile à copier ou à soumettre.

**Résultat :**

- J'obtiens la signature du challenge « fader scrub chins divvy amass », produite avec ma clé privée RSA, puis convertie en hexadécimal pour pouvoir l'utiliser plus facilement.

**FLAG 9 : Génération d'une requête de certificat (CSR)**

**Objectif :**

- Générer une requête de certificat pour ma clé privée RSA.

**Méthode :**

- Lancer le code « provide_certificate.py » :
    - Exécuter une commande OpenSSL qui crée une nouvelle requête de certificat.
    - Utiliser la clé privée ./Keys/PrivateKeys/sk_atrium.pem pour associer la requête à ma paire de clés RSA.
    - Définir directement le sujet du certificat avec /CN=AdrienPanguel.
    - Désactiver les questions interactives pour automatiser la génération.
    - Stocker la requête générée dans ./Certificates/certificate_atrium.csr.pem.

**Explications du code :**

- La commande OpenSSL utilisée est openssl req, qui sert à créer une requête de certificat.
- L'option -new indique que l'on veut générer une nouvelle CSR (Certificate Signing Request).
- L'option -batch évite les questions interactives, ce qui permet une exécution automatique du script.
- L'option -subj /CN=AdrienPanguel définit directement le sujet du certificat, ici le Common Name.

**Résultat :**

- J'obtiens une requête de certificat ./Certificates/certificate_atrium.csr.pem associée à ma clé privée RSA et portant le sujet CN=AdrienPanguel.

**FLAG 10 : Broadcast & Chiffrement hybride (AES+RSA) avec une clé de session**

**Objectif :**

- Chiffrer un Plaintext à l'aide d'une clé de session.
- Faire un broadcast de la session-key et du Ciphertext chiffré avec.

**Méthode :**

- Lancer le code « build_session_key.py » :
    - Définir le Plaintext « AdrienPanguel123 » qui servira de clé de session.
    - Exécuter une commande OpenSSL qui chiffre ce Plaintext avec la clé publique ./Keys/PublicKeys/pk_broadcast.pem.
    - Convertir la sortie binaire en hexadécimal.
    - Supprimer les retours à la ligne pour obtenir une chaîne compacte.
    - Stocker le résultat dans ./Keys/session_key.pem.
- Lancer ensuite le code « encrypt_plaintext_session_key.py » :
    - Définir le Plaintext test dans une variable plaintext.
    - Définir la clé de session AdrienPanguel123 dans une variable password.
    - Appeler la fonction encrypt(plaintext, password) pour chiffrer le message.
    - Stocker le résultat chiffré dans la variable plaintext.
    - Afficher le Ciphertext obtenu.

**Explications du code :**

- Dans build_session_key.py, la commande OpenSSL utilisée est pkeyutl -encrypt -pubin, ce qui permet de chiffrer la future clé de session avec une clé publique.
- L'option -inkey ./Keys/PublicKeys/pk_broadcast.pem indique la clé publique utilisée pour protéger la clé de session.
- Le script supprime les sauts de ligne avec replace("\n", "").replace(" ", "") afin d'obtenir une chaîne compacte, plus facile à stocker ou transmettre.

**Résultat :**

- J'obtiens :
    - Une clé de session protégée par chiffrement asymétrique et stockée dans ./Keys/session_key.pem
    - Un Ciphertext correspondant au Plaintext test, chiffré avec la clé de session AdrienPanguel123.

**FLAG 11 : Signature de la clé publique d'un ami**

**Objectif :**

- Signer la clé publique d'un ami avec ma clé privée RSA.
- Faire signer ma clé publique par 2 camarades avec leur clé privée.

**Contexte :**

- J'ai :
    - La clé publique de mon collègue.

**Méthode :**

- Lancer le code « sign_friends.py » :
    - Exécuter une commande OpenSSL qui calcule un haché SHA-256 du fichier ./Keys/PublicKeys/pk_friend1.pem et le signe avec ma clé privée RSA.
    - Stocker la signature binaire dans ./Binary/sign_friend1.bin.
    - Convertir la signature binaire en hexadécimal.
    - Supprimer les retours à la ligne pour obtenir une chaîne compacte.
    - Stocker le résultat dans ./Hexa/sign_friend1.hex.

**Explications du code :**

- L'option -sha256 précise que la fonction de hachage utilisée est SHA-256.
- Le script enlève les sauts de ligne de la sortie hexadécimale avec replace("\n", "") pour obtenir une chaîne continue, plus facile à copier ou à transmettre.

**Résultat :**

- J'obtiens la signature de la clé publique ./Keys/PublicKeys/pk_friend1.pem, produite avec ma clé privée RSA, puis convertie en hexadécimal dans ./Hexa/sign_friend1.hex.

**FLAG 12 : Shared modulus Attack (RSA) avec fuite de Private Key**

**Objectifs :**

- Retrouver les clés privées associées à deux clés publiques RSA qui partagent le même module, sachant que je connais une des deux clés privées.

**Contexte :**

- Je connais :
  - Les clés publiques $(e_A, N_A)$ et $(e_B, N_B)$.
  - L'exposant privée $d_A$ de l'individu A.

**Attaque :**

- Connaissant $e_A$ et $d_A$, je peux calculer :

$$e_A \cdot d_A \equiv 1 \; mod \; \varphi(N_A)$$

- En posant :

$$M = e_A \cdot d_A - 1$$

- On peut réécrire :

$$M = k \cdot \varphi(N_A) = ed - 1 = 2^t \cdot r, avec \; r \; impair$$

- On choisit ensuite un entier aléatoire g entre 2 et $(N_A - 2)$, puis on calcule :

$$x = g^r \, mod N_A$$

- Ensuite on regarde la suite :

$$x, x^2, x^{2^2}, \dots, x^{2^t}$$

- C'est-à-dire :

$$g^r, g^{2r}, \dots, g^{2^{tr}} \; mod \; N_A$$

- Comme M est un multiple de $\varphi(N_A)$, on a souvent :

$$g^M \equiv 1 \bmod N_A$$

- Donc, en remontant la chaîne des carrés, on cherche le moment où on passe d'une valeur non triviale à 1, dont le cas intéressant est :

$$\begin{cases} y^2 \equiv 1 \bmod N_A \\ y \neq \pm 1 \; mod \; N_A \end{cases}$$

$$\Rightarrow y^2 \equiv 1 \bmod N_A \Rightarrow y^2 - 1 \equiv 0 \; mod \; N_A \Rightarrow (y-1)(y+1) \equiv 0 \bmod N_A$$

$$\Rightarrow N_A | (\mathrm{y} - 1)(\mathrm{y} + 1)$$

- On sait que $y \neq \pm 1 \; mod \; N_A$, donc que y est une racine carrée non triviale de 1 modulo $N_A$, c'est-à-dire que comme $N_A$ ne divise pas $(y-1)$ tout seul ou $(y+1)$ tout seul, alors p divise l'un et q divise l'autre, dans un de ces cas par exemple :

$$\begin{cases} \gcd(y-1, N_A) = p \\ q = \dfrac{N_A}{p} \end{cases}$$

- On a retrouvé p et q !

**Méthode :**

- Lancer le code « shared_modulus_and_private_key.py » :

- Charger les deux clés publiques ./Keys/PublicKeys/pk_atriumA.pem et ./Keys/PublicKeys/pk_atriumB.pem.
- Extraire leurs paramètres publics $n_A$, $e_A$ et $n_B$, $e_B$.
- Récupérer la valeur de la clé privée dB sous forme hexadécimale, puis la convertir en entier.
- Vérifier que les deux clés publiques partagent bien le même module RSA, c'est-à-dire que $n_A == n_B$.
- Utiliser les valeurs eB, dB et nB pour factoriser le module commun n et retrouver ses deux facteurs premiers p et q.
- Calculer :

$$phi(n) = (p - 1)(q - 1)$$

- Reconstruire ensuite la clé privée A avec p, q et $e_A$.
- Reconstruire aussi la clé privée B avec p, q et $e_B$.
- Sauvegarder les deux clés privées dans :
  - ./Keys/PrivateKeys/sk_atriumA.pem
  - ./Keys/PrivateKeys/sk_atriumB.pem

**<u>Explications du code :</u>**

- La variable d_hex_B contient l'exposant privé $d_B$ de la clé B, écrit en hexadécimal, et la ligne int(d_hex_B, 16) le convertit en entier.
- La fonction factor_n_from_ed(e, d, n) exploite la relation fondamentale RSA $e * d \equiv 1 \; mod \; \varphi(n)$. Autrement dit, $k = e * d - 1$ est un multiple de $\varphi(n)$, ce qui permet de retrouver un facteur non trivial de n avec une méthode probabiliste fondée sur des calculs de puissances modulaires et de PGCD.
- La fonction modinv(a, m) calcule un inverse modulaire grâce à l'algorithme d'Euclide étendu. Elle sert notamment à recalculer $d = e^{-1} \; mod \; \varphi(n)$.
- La fonction save_priv_key(p, q, e, path) reconstruit tous les paramètres d'une clé privée RSA :
- Ces paramètres sont injectés dans rsa.RSAPrivateNumbers(...), puis exportés au format PEM PKCS8 sans chiffrement avec private_bytes(...).

**<u>Résultat :</u>**

- J'obtiens les deux clés privées RSA :
  - ./Keys/PrivateKeys/sk_atriumA.pem
  - ./Keys/PrivateKeys/sk_atriumB.pem

**FLAG 13 : Shared Modulus Attack (RSA)**

**Objectif :**

- Retrouver le Plaintext qui a été chiffré par deux individus avec la méthode RSA utilisant le même modulus N.

**Rappel (construction de la clé privée en RSA) :**

- Generate two large primes p and q with $\mathrm{p} \neq \mathrm{q}$.
- Compute:
$$\mathrm{N} = \mathrm{p} \times \mathrm{q}$$
- Compute Euler's totient:
$$\varphi(\mathrm{N}) = (\mathrm{p} - 1) \times (\mathrm{q} - 1)$$
- Choose a random public exponent e (common choice is $\mathrm{e} = 65537$) such that:
$$\begin{cases} 1 < \mathrm{e} < \varphi(\mathrm{N}) \\ \gcd(\mathrm{e}, \varphi(\mathrm{N})) = 1 \end{cases}$$
- Compute the unique integer d such that:
$$\begin{cases} 1 < \mathrm{d} < \varphi(\mathrm{N}) \\ \mathrm{d} \times \mathrm{e} \equiv 1 \bmod \varphi(\mathrm{N}) \end{cases}$$
- The public key is (N, e).
- The private key is (d, p, q).

**Contexte :**

- Je connais :
  - Les clés publiques $(e_1, N_1)$ et $(e_2, N_2)$.
  - Les chiffrés du même message $c_1$ et $c_2$.
- Je sais que :
  - Le module est le même : $N_1 = N_2 = N$.
  - Le message m est tel que $0 \leq m < N$.

**Attaque :**

- Vérifier que $e_1$ et $e_2$ sont premier entre eux : $\gcd(e_1, e_2) = 1$.
- D'après le théorème de Bézout, comme $e_1$ et $e_2$ sont premier entre eux :
$$u \cdot e_1 + v \cdot e_2 = 1$$
- Mettre le chiffré $c_1$à la puissance $u$ et le chiffré $c_2$à la puissance $v$ :
$$\begin{cases} c_1^u = m^{e_1 u} \; mod \; N = m^{e_1 \cdot u} \; mod \; N \\ c_2^v = m^{e_2 v} \; mod \; N = m^{e_2 \cdot v} \; mod \; N \end{cases}$$
- Attention, comme soit a soit b est négatif (supposons $a < 0$), je ne peux pas calculer $c_1^a$ directement, donc je dois passer par l'inverse modulaire :
$$c_1^{-|a|} = (c_1^{-1})^{|a|} \; mod \; N$$
- Et l'inverse modulaire de $c_1$ existe car $gcd(c_1, N) = 1$, sinon on aurait
- factorisé N, ce qui serait encore mieux.
- Je multiplie ces 2 chiffrés $c_1^u$ et $c_2^v$ entre eux :
$$c_1^u \cdot c_2^v = m^{e_1 \cdot u} \cdot m^{e_2 \cdot v} \; mod \; N = m^{e_1 \cdot u + e_2 \cdot v} \; mod \; N = m \; mod \; N = m$$
- J'ai retrouvé le message m !

**Remarque :**

- Quand $e_1$et $e_2$ ne sont pas premiers entre eux, je pose :
$$d = \gcd(e_1, e_2) > 1$$

- Tel que :

$$\begin{cases} e_1 = d \cdot a \\ e_2 = d \cdot b \\ \gcd(a, b) = 1 \end{cases}$$

- Comme $a$ et $b$ sont premiers entre eux, il existe $(u, v)$ tels que :

$$u \cdot a + v \cdot b = 1$$
$$\Rightarrow u \cdot e_1 + v \cdot e_2 = d$$

- À partir des chiffrés, on calcule :

$$X = c_1^u \cdot c_2^v \bmod N = m^{e_1 u} \cdot m^{e_2 v} \bmod N = m^{e_1 \cdot u + e_2 \cdot v} \; mod \; N = m^d \; mod \; N = m^d$$

- Il suffit de prendre la racine $d$-ième entière :

$$m = \sqrt[d]{X}$$

- J'ai retrouvé le message m !

**<u>Méthode :</u>**

- Lancer le code « shared_modulus.py » :
  - Charger les deux clés publiques RSA encodées en PEM.
  - Extraire leurs paramètres publics n1, e1 et n2, e2.
  - Convertir les deux ciphertexts hexadécimaux en entiers avec hex_to_int().
  - Vérifier que n1 == n2 et que gcd(e1, e2) == 1.
  - Utiliser l'algorithme d'Euclide étendu pour trouver a et b tels que :
    - a*e1 + b*e2 = 1
  - Reconstituer le message m avec la formule de la common modulus attack.
  - Convertir l'entier obtenu en bytes, puis en texte.
    - Rechercher dans le texte le motif « Nouveau digicode de la salle RC-25 : … »
  - Extraire le digicode trouvé et le stocker dans ./Texts/digicode.txt.

**<u>Explications du code :</u>**

- La fonction load_pubkey_numbers(pem) charge une clé publique PEM et extrait n et e. Elle essaie d'abord avec la bibliothèque cryptography, puis utilise Crypto.PublicKey.RSA en secours si nécessaire.
- La fonction hex_to_int(h) convertit un ciphertext hexadécimal en entier Python avec int(h, 16).
- La fonction egcd(a, b) implémente l'algorithme d'Euclide étendu et renvoie le PGCD ainsi que les coefficients de Bézout.
- La fonction modinv(a, m) calcule l'inverse modulaire de $a \; mod \; m$. Elle est nécessaire lorsque l'un des exposants de Bézout est négatif.
- La fonction mod_pow_signed(base, exp, mod) gère ce cas :
  - si $exp \geq 0$, elle calcule $base^{exp} \; modulo \; (mod)$
  - sinon, elle calcule l'inverse modulaire de base, puis élève cet inverse à la puissance -exp.
  - La ligne « m = (mod_pow_signed(c1, a, n1) * mod_pow_signed(c2, b, n1)) % n1 » applique directement la formule de l'attaque.
- Le script utilise ensuite une expression régulière avec re.search(…) pour extraire uniquement la valeur du digicode.

**<u>Résultat :</u>**

- J'obtiens le message en clair commun aux deux ciphertexts RSA.

**FLAG 14 : Attaque par recherche exhaustive sur une seed trop petite**

**Objectif :**

- Montrer qu'un système qui utilise un chiffrement correct en apparence, peut être cassé si la clé et l'IV proviennent d'un secret minuscule.

**Contexte :**

- Je connais :
  - Un Ciphertext.
  - Un IV cible de 128 bits : 96c2089c5c89644420794597e1eba19b.
- Je sais que :
  - La clé AES-128 et l'IV sont dérivés à partir d'une seed de 16 bits seulement.
  - Il n'existe donc que 65536 seeds possibles, ce qui rend une recherche exhaustive réaliste

**Attaque :**

- Essayer toutes les seeds possibles sur 16 bits.
- Pour chaque seed :
  - Dériver 32 octets avec la fonction key_expansion().
  - Prendre les 16 premiers octets comme clé AES K.
  - Prendre les 16 derniers octets comme IV.
- Comparer l'IV dérivé avec l'IV cible connu.
- Si l'IV correspond, alors la seed est la bonne, donc la clé K aussi.
- Utiliser ensuite K et l'IV trouvé pour déchiffrer le Ciphertext AES-128-CBC.

**Méthode :**

- Essayer toutes les seeds possibles en dérivant pour chaque seed une clé K et un IV.
- Si l'IV dérivé ets identique à l'IV connu, alors la seed est la bonne, donc la clé K aussi.
- Utiliser K et l'IV pour déchiffrer le ciphertext donné.
- Lancer le code « key_expansion_seed.py » :
  - Lire le fichier ./Texts/ciphertext_key_expansion_seed.txt.
  - Nettoyer son contenu Base64 en supprimant les espaces et retours à la ligne.
  - Décoder le Base64 pour obtenir les octets binaires du ciphertext.
  - Convertir l'IV cible hexadécimal en octets avec bytes.fromhex().
  - Tester les deux interprétations possibles de la seed : big-endian puis little-endian.
  - Pour chaque seed, appeler key_expansion(seed) pour dériver 32 octets.
  - Séparer ces 32 octets :
    - K = km[:16]
    - IV = km[16:32]
  - Quand l'IV obtenu est le même que l'IV cible, récupérer la seed, la clé K en hexadécimal, l'IV, et l'endianness utilisée.
  - Déchiffrer le ciphertext avec openssl enc -d -aes-128-cbc -K … -iv ….
  - Écrire le plaintext obtenu dans ./Texts/key_expansion_seedTEST.txt.
  - Appliquer ensuite une heuristique looks_plausible() pour vérifier si le plaintext ressemble à un texte crédible.
  - Déchiffrer le Ciphertext avec la clé K trouvée.

**Explications du code :**

- La fonction key_expansion(seed) part d'un état initial égal à la seed, puis applique SHA-256 huit fois de suite. À chaque tour, elle concatène les 4 premiers octets du hash au résultat final. On obtient ainsi 8 × 4 = 32 octets.
- La fonction openssl_decrypt_aes128cbc() appelle OpenSSL avec :
    - -d pour déchiffrer,
    - -aes-128-cbc pour l'algorithme,
    - -K pour passer la clé en hexadécimal,
    - -iv pour passer l'IV en hexadécimal.
- La fonction looks_plausible(pt) essaie de décoder le plaintext en UTF-8, puis vérifie soit la présence de mots-clés comme FLAG, digicode, Uglix, {, }, soit un fort taux de caractères imprimables.

**Résultat :**

- J'obtiens la seed correcte, la clé AES K, et l'IV correspondant.
- J'ai déchiffré le Ciphertext donné en un Plaintext.

**FLAG 15 : Attaque de Håstard (petit exposant publique commun)**

**Objectif :**

- Retrouver le Plaintext message $m$ qui a été chiffré par trois individus avec la méthode RSA utilisant le même petit exposant $e$ (passer par le théorèe chinois des restes).

**Contexte :**

- Je connais :
  - Les chiffrés de 3 individus :
$$\begin{cases} c_1 \equiv m^e \ mod \ N_1 \\ c_2 \equiv m^e \ mod \ N_2 \\ c_3 \equiv m^e \ mod \ N_3 \end{cases}$$
  - Leurs clés publiques $(e_1, N_1)$, $(e_2, N_2)$, et $(e_3, N_3)$.
- Je sais que :
  - Le message m est tel que $0 \leq m < N$.

**Attaque :**

- Mathématiquement, je cherche un entier $X$ tel que :
$$\begin{cases} X \equiv c_1 \bmod N_1 \\ X \equiv c_2 \bmod N_2 \\ X \equiv c_3 \bmod N_3 \end{cases}$$
- Comme :
$$c_i \equiv m^e \bmod N_i$$
- Cela revient à chercher un $X$ vérifiant :
$$\begin{cases} X \equiv m^e \bmod N_1 \\ X \equiv m^e \bmod N_2 \\ X \equiv m^e \bmod N_3 \end{cases}$$
- Le théorème chinois des restes dit que si $N_1$, $N_2$ et $N_3$ sont deux à deux premiers entre eux, alors ce système admet une solution unique modulo $M = N_1 N_2 N_3$.
- Autrement dit, on peut reconstruire un entier $X$ qui satisfait les trois congruences tel que :
$$0 \leq X < M$$
- Donc :
$$X \equiv m^e \bmod N_1 N_2 N_3 \equiv m^e \ mod \ M = m^e$$
- Pour calculer ce $X$, on pose :
$$\begin{cases} M = N_1 N_2 N_3 \\ M_i = \dfrac{M}{N_i} \\ u_i \equiv M_i^{-1} \bmod N_i \end{cases}$$
- D'où la formule explicite pour calculer $X$ :
$$X \equiv c_1 M_1 u_1 + c_2 M_2 u_2 + c_3 M_3 u_3 \bmod M = m^e \ mod \ M = m^e$$
- Je peux alors calculer :
$$m = \sqrt[e]{X}$$
- J'ai retrouvé le message m !

**Preuve de la validité du théorème chinois des restes :**

- A partir du système de congruences suivant :

$$\begin{cases} X \equiv m^e \bmod N_1 \\ X \equiv m^e \bmod N_2 \\ X \equiv m^e \bmod N_3 \end{cases} \Rightarrow \begin{cases} (X - m^e) \equiv N_1 \\ (X - m^e) \equiv N_2 \\ (X - m^e) \equiv N_3 \end{cases}$$

- Posons :

$$z = X - m^e$$

- On sait :

$$\begin{cases} N_1 \mid z \\ N_2 \mid z \\ N_3 \mid z \end{cases}$$

- Je veux montrer que :

$$N_1 N_2 N_3 \mid z$$

- **Étape 1 : combiner $N_1$et $N_2$**
- Comme $N_1 \mid z$, il existe $a \in \mathbb{Z}$ tel que :

$$z = N_1 a$$

- Mais comme aussi $N_2 \mid z$, donc :

$$N_2 \mid N_1 a \Rightarrow N_2 | N_1 \; ou \; N_2 | a$$

- Or :

$$\gcd(N_1, N_2) = 1$$

- Donc $N_2$ ne divise pas $N_1$, donc :

$$N_2 \mid a$$

- Donc il existe $b \in \mathbb{Z}$ tel que :

$$a = N_2 b$$

- En remplaçant dans $z = N_1 a$, j'obtiens :

$$z = N_1 N_2 b$$

- Donc :

$$N_1 N_2 \mid z$$

- **Étape 2 : ajouter $N_3$**
- J'ai maintenant :

$$\begin{cases} N_1 N_2 \mid z \\ N_3 \mid z \end{cases}$$

- Je refais exactement le même raisonnement, comme $N_3 \mid z$, il existe $c \in \mathbb{Z}$ tel que :

$$z = N_3 c$$

- Mais comme aussi $N_1 N_2 \mid z$, donc :

$$N_1 N_2 \mid N_3 c \Rightarrow N_1 N_2 | N_3 \; ou \; N_1 N_2 | c$$

- Or $N_1, N_2, N_3$sont deux à deux premiers entre eux, donc $N_1 N_2$ ne divise pas $N_3$, alors :

$$N_1 N_2 \mid c$$

- Donc il existe $d \in \mathbb{Z}$ tel que :

$$c = N_1 N_2 d$$

- En remplaçant dans $z = N_3 c$, j'obtiens :

$$z = N_1 N_2 N_3 d$$

- Donc :

$$N_1 N_2 N_3 \mid z$$

- Donc :

$$N_1 N_2 N_3 \mid (X - m^e)$$

- Autrement dit :

$$X \equiv m^e \bmod N_1 N_2 N_3$$

**<u>Méthode :</u>**

- Lancer le code « decrypt_ciphertext_distributeur.py » :
    - Définir les trois ciphertexts c1_hex, c2_hex, c3_hex en hexadécimal.
    - Définir les trois modules RSA n1_hex, n2_hex, n3_hex en hexadécimal.
    - Convertir toutes ces valeurs hexadécimales en entiers avec int(..., 16).
    - Appliquer le théorème des restes chinois avec la fonction crt(c_list, n_list) pour reconstruire m_cubed, c'est-à-dire $m^3$.
    - Calculer la racine cubique entière de cette valeur avec integer_cbrt(m_cubed).
    - Convertir l'entier obtenu en hexadécimal puis en bytes.
    - Rechercher la fin du padding RSA en séparant sur l'octet nul 0x00.
    - Extraire la dernière partie, qui correspond au message utile.
    - Tenter de décoder ce message en UTF-8 pour obtenir le Plaintext lisible.

**Explications du code :**

- La fonction crt(c_list, n_list) applique le théorème des restes chinois :
- La fonction integer_cbrt(n) calcule la racine cubique entière par dichotomie, en cherchant l'entier dont le cube redonne la valeur reconstruite.
- La ligne m_cubed = crt([c1, c2, c3], [n1, n2, n3]) reconstruit donc $m^3$.
- La ligne m_int = integer_cbrt(m_cubed) retrouve ensuite m.
- La fonction get_printable_message(n) convertit m en hexadécimal, ajoute un zéro en tête si nécessaire pour conserver un nombre pair de chiffres, et transforme cela en bytes avec binascii.unhexlify().
- Cette fonction cherche ensuite l'octet 0x00 pour repérer la fin d'un éventuel padding RSA PKCS#1 v1.5, puis garde la dernière partie comme message utile.

**Résultat :**

- J'obtiens le Plaintext commun aux trois ciphertexts RSA.

**FLAG 16 : Signature aveugle (RSA)**

**Objectif :**

- Obtenir la signature RSA finale d'un message en faisant signer un message aveugle par le directeur.

**Contexte :**

- Je connais :
  - Le module RSA du directeur : N_hex.
  - Le message à faire signer aveuglément « I, the lab director, hereby grant AdrienPanguel permission to take the BiblioDrone-NG. »
- Je sais que :
  - La signature renvoyée par le laptop porte sur un message aveuglé.
  - Pour retrouver la vraie signature, il faut retirer l'aveuglement en multipliant par l'inverse modulaire de x modulo N.

**Attaque :**

- Le serveur du directeur ne fait pas une vraie signature RSA complète, c'est-à-dire qu'il se comporte comme un oracle RSA brut, car il prend juste un entier en entrée et calcule $m^d mod N$, or RSA brut est malléable car la signature d'un produit est liée au produit des signatures.
- On commence par construire correctement le bloc $M$correspondant au message que je veux faire signer avec la signature du directeur, avec l'encodage PKCS#1 v1.5 attendu par OpenSSL.
- Ensuite, au lieu d'envoyer directement $M$, je choisis un entier aléatoire $x$ inversible modulo $N$, et $e$ du directeur, j'envoie :
$$M' = M \cdot x^e \ mod N$$
- Le serveur signe alors cette valeur masquée et me renvoie :
$$(M')^d = (M \cdot x^e)^d \equiv M^d \cdot x \bmod N$$
- On obtient donc la vraie signature $M^d$, mais multipliée par le facteur $x$.
- Comme on connaît $x$, on calcule son inverse modulo $N$, puis on retire le masque :
$$S = (M')^d \cdot x^{-1} \equiv M^d \bmod N$$
- Le résultat $S$est alors exactement la signature RSA valide du message voulu.

**Méthode :**

- Lancer le code « blinding_plaintext.py » :
  - Lire le modulus N de la clé publique du directeur avec OpenSSL.
  - Fixer l'exposant public à $e = 65537$.
  - Définir le message à faire signer.
  - Calculer son hash SHA-256 avec hashlib.sha256(msg).digest().
  - Ajouter l'identifiant ASN.1 standard de SHA-256.
  - Construire le bloc PKCS#1 v1.5 complet en binaire.
  - Convertir ce bloc en entier M.
  - Calculer la version aveuglée :
$$M_blind = (M * pow(x, e, N)) \% N \ avec \ x = 2.$$
  - Afficher la donnée aveuglée à envoyer au directeur.
- Lancer ensuite le code « obtain_sign.py » :
  - Définir la signature aveuglée S_blind_hex.
  - La convertir en entier avec int(S_blind_hex, 16).
  - Définir le modulus N_hex du directeur et le convertir en entier avec int(N_hex, 16).

- Calculer l'inverse modulaire de $x \equiv 2 \; mod \; N$ avec pow(x, -1, N).
- Retirer l'aveuglement en calculant :

$$S_final \; = \; (S_blind \; * \; pow(x, -1, N)) \; \% \; N$$

- Afficher la signature finale en hexadécimal.

**<u>Explications du code :</u>**

- Dans blinding_plaintext.py, la commande OpenSSL « openssl rsa -pubin -in ./Keys/PublicKeys/pk_director.pem -noout -modulus » permet de récupérer le modulus N de la clé publique du directeur.
- Le script construit un vrai bloc de signature PKCS#1 v1.5 :

$$00 \; 01 \; FF \ldots FF \; 00 \; || \; ASN.1(SHA - 256) \; || \; hash(message)$$

- La taille du bloc est ajustée à la taille du modulus grâce au calcul de k = (N.bit_length() + 7) // 8.
- Dans obtain_sign.py, la fonction Python pow(x, -1, N) calcule directement l'inverse modulaire de x modulo N.
- La ligne « S_final = (S_blind * pow(x, -1, N)) % N » reconstitue donc la vraie signature RSA du message original.

**<u>Résultat :</u>**

- J'obtiens la signature RSA finale non aveuglée du message destiné au robot.

**FLAG 17 : Interpolation polynomiale de Lagrange sur des MACs**

**Objectifs :**

- Retrouver les coefficients d'un polynôme P(X) de degré 8, qui a donc 9 coefficients inconnus, à partir de 9 couple points/images.

**Contexte :**

- Je connais :
  - Un corps fini défini par le module premier $P = 2^{64} - 59$.
  - Les 9 premiers flags, chacun représenté par une chaîne hexadécimale de 64 caractères.
- Je sais que :
  - Chaque MAC est découpé en 4 blocs hexadécimaux de 16 caractères : X A B C
  - Ces blocs servent à construire trois ensembles de points : (X, A), (X, B), (X, C).

**Attaque :**

- Certaines firmware-keys sont produites par un polynôme, donc connaissant 9 firmware-keys, comme ce polynôme est de degré 8, alors il possède 9 coefficients inconnus, je peux le reconstruire.
- Les 9 premières clés connues te donnent ces 9 points, car pour chaque entrée $x_i$, tu connais la valeur correspondante $P(x_i)$. Avec ces 9 couples $(x_i, P(x_i))$, ton script fait une interpolation polynomiale et reconstitue entièrement le polynôme $R(X)$.
- Une fois le polynôme retrouvé, tu peux l'évaluer en un point particulier, ici $X = 0$. Le script calcule donc :

$$R(0) = 3227412088583349083$$

- Puis l'écrit en hexadécimal :

0x2cca1231ebcfe35b

- Cette valeur correspond à la firmware update key attendue par la borne. On enlève simplement le préfixe 0x pour saisir :

$$2cca1231ebcfe35b$$

**Méthode :**

- Lancer le code « interpolation.py » :
  - Définir le module premier $P = 2^{64} - 59$.
  - Stocker les 9 premiers flags dans une liste flags.
  - Pour chaque MAC, parse_mac(mac) le découpe en quatre entiers : X, A, B, C.
  - Construire trois listes de points avec pts_R.append((X, A)), pts_S.append((X, B)), et pts_T.append((X, C)).
  - Éliminer les éventuels doublons sur X avec l'ensemble seen.
  - Utiliser la fonction lagrange_eval_at_zero(points, p) pour évaluer le polynôme interpolé en 0.
  - Calculer ainsi le polynôme R0 = lagrange_eval_at_zero(pts_R[:9], P).
  - Le script prévoit aussi le calcul de S0 avec 17 points, et de T0 avec 25 points.
  - Afficher les résultats en entier puis en hexadécimal avec hex(R0), hex(S0), hex(T0).

**Explications du code :**

- La fonction parse_mac(mac) vérifie d'abord que le MAC a bien une longueur de 64 caractères hexadécimaux, puis le découpe en quatre blocs de 16 hex digits, chacun converti en entier.
- La fonction egcd(a, b) implémente l'algorithme d'Euclide étendu. Elle sert à calculer les coefficients de Bézout.
- La fonction modinv(a, p) calcule l'inverse modulaire de a modulo p.

- La fonction lagrange_eval_at_zero(points, p) applique directement la formule :

$$L_{i(0)} = \prod_{\{j \neq i\}} \frac{-x_j}{x_i - x_j} \ mod\ p$$

- Puis somme les termes $y_i * L_i(0)\ mod\ p$.

**<u>Résultat :</u>**

- J'obtiens R(0), c'est-à-dire la valeur en zéro du polynôme reconstruit à partir des couples (X, A).